

Mutual Exclusion Mechanism

High Performance
Computing

Master in Applied Artificial
Intelligence

Nuno Lopes

Parallel programming challenge

- ▶ Develop a program that increments a global variable N times.
- ▶ Sequential version relies on a single thread.
- ▶ Parallel version must use more than 1 thread to increment a global counter variable.
- ▶ Correctness: the result of both applications must always be the same
- ▶ Performance: the parallel version is expected to be n times faster for n threads.

Implementation Exercise

Sequential Version

- ▶ Implement code that increments a global variable.
 - ▶ consider parameter `#iterations`
- ▶ Time measurement serves for comparison.

Parallel Version

- ▶ Implement parallel version that
- ▶ Make use of threads;
- ▶ Allow configuring the number of threads, and distribute the cycle size among the threads.

Race Condition and Critical Section

- ▶ A **race condition** arises when several threads try to access the same shared variable, for reading and/or writing, at the same time.
- ▶ The characteristics of the hardware sometimes cause inconsistencies to arise in the values of these variables when they are subject to read/write operations.
- ▶ A **critical section** is the code executed by several threads that accesses a shared variable, and that must be "protected" from simultaneous access by several threads, or it will cause race conditions.

Intel Inspector Documentation

Race Conditions Are Difficult to Diagnose

They occur only occasionally and are difficult to reproduce

Correct				Incorrect			
Thread 1	Thread 2		Shared Counter	Thread 1	Thread 2		Shared Counter
			0				0
Read count		←	0	Read count		←	0
Increment			0		Read count	←	0
Write count		→	1	Increment			0
	Read count	←	1		Increment		0
	Increment		1	Write count		→	1
	Write count	→	2		Write count	→	1

Mutual Exclusion Mechanism

- ▶ In order to prevent multiple threads from executing critical sections simultaneously, there must be mechanisms to ensure that only one thread can, in turn, access the same resources shared between them.
- ▶ These mechanisms are called "mutual exclusion mechanisms".

Solution to Get it Correct in the Parallel Version

- ▶ It is necessary to use mutual exclusion mechanism.
- ▶ Each thread cannot freely access a shared variable.
- ▶ Identify:
 1. Shared variable;
 2. Thread code that accesses the variable;
 3. Rotate this code by a mutual exclusion mechanism.

Mutual Exclusion Mechanisms in OpenMP

Variable Visibility: *shared, private*

- ▶ Each thread accesses existing program variables as well as new variables.
- ▶ Variables that already exist before the parallel section, are shared by all threads, **shared**.
- ▶ New variables, internal to the cycle, only exist in each thread individually, they are only visible to the thread itself: **private**.
- ▶ If a thread changes the value of a shared variable, this new value will be visible to all other threads.
- ▶ Proven variables have a different value for each thread.

Critical Directive

```
# pragma omp critical
```

```
<code>
```

- ▶ Directive indicates that the following code is a "critical section" and is protected by a mutual exclusion mechanism between all threads.
- ▶ It means that it can only be executed by one thread at a time and not simultaneously.

Accumulation of Results in Shared Global Variable

- ▶ The use of the critical clause allows you to guarantee the correctness of the program.
- ▶ However, it creates a serialization of execution, i.e. the program behaves in a sequential way, which makes you lose the advantage of parallelism!
- ▶ Sometimes, critical sections just want to collect a result local to each thread, to be "accumulated" into a global value.

Reduction Clause

`reduction(<operator>: <variable list>)`

- ▶ The **reduction** clause allows you to use a shared global variable, as if it were private, during thread execution.
- ▶ At the end of the execution, the private value of each thread is accumulated ("reduced") by the operator, to the (single) global variable.
- ▶ This is equivalent to using an auxiliary private variable to accumulate the thread's internal values, and then integrating them into the shared global value.